

Praktikum 1 - Matakuliah Pilihan Web (API Development)

Program Studi: Teknik Informatika

Lakukan praktikum dibawah ini, dan buat screenshot untuk pembuktian mengerjakan setiap poin dengan mengisi tabel dibawah, kemudian tunjukan hasil akhir dari men-share repository github yang telah dibuat.

A. Setup Laravel dan Mysql

1. Buat project laravel baru dengan composer dengan nama laravel_api (Gunakan versi [12.x](#) atau yang lebih baru)
2. Mempersiapkan database dan buat database baru bernama dblaravelapi
3. Update .env dan hubungkan dengan nama database dblaravelapi
4. Lakukan migrasi dengan menggunakan php artisan migrate
5. Pastikan laravel sudah terhubung dengan tabel dengan mengecek tabel yang dibuat laravel dalam mysql

B. Membuat Controller

1. Buatlah sebuah controller dengan nama Api/ProductController
php artisan make:controller Api/ProductController
(Pastikan ada di dalam folder Api)
Pada bootstrap/app.php tambahkan setelah code web: __DIR__
api: __DIR__.'../routes/api.php',

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'../routes/web.php',
        api: __DIR__.'../routes/api.php',
        commands: __DIR__.'../routes/console.php',
        health: '/up',
    );
```

2. Tambahkan Routes baru di router/api.php
Route::get('/products', [ProductController::class, 'index']);
(Apabila file api.php belum ada, clone web.php dan hapus routes "")
3. Test API dengan mengunjungi localhost:8000/api/products

C. Gunakan database dan Model

1. Pastikan database sudah terkoneksi dengan laravel.
2. Buatlah sebuah tabel dengan nama tblproducts dengan field (id, nama, deskripsi, foto, dan harga) untuk tipe data menyesuaikan.

3. Tambahkan minimal 10 data untuk produk dengan nama2 product elektronik seperti tv, kulkas, ac, dan lain sebagainya.
4. Buat model bernama Product
php artisan make:model Product
5. Hubungkan model Product dengan tblproduct dengan mengisi pada class Product

```
class Product extends Model
{
    protected $table = 'tblproducts';
    protected $primaryKey = 'id';
    public $timestamps = false;

    // Field yang boleh diisi
    protected $fillable = [
        'nama',
        'deksripsi',
        'foto',
        'harga',
    ];
}
```

6. Pada Product controller ubah dalam fungsi index, dapatkan data dari model Product Dengan menambahkan \$products = Product::all(); Dan lakukan return data => \$products

D. Membuat CRUD standar API Resource

1. Ubah api.php dengan api Resource yang sesuai standar laravel

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\ProductController;

// Route::get('/products', [ProductController::class, 'index']);

Route::apiResource('products', ProductController::class);
```

Dengan apiResource kamu tidak perlu menambahkan routes satu persatu untuk ProductController seperti create, update, delete, search dan view. Namun ada syarat penamaan fungsi agar routes ini bekerja yaitu **index, show, store, update, dan destroy**.

- Ubah ProductController untuk fungsi-fungsi berikut **index**, **show**, **store**, **update**, dan **destroy**.

```
// ◆ GET /products
public function index()
{
    $products = Product::all();

    return response()->json([
        'status' => true,
        'data' => $products
    ]);
}
```

Gambar 04: Fungsi Index pada ProductsController

```
// ◆ GET /products/{id}
public function show($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    return response()->json([
        'status' => true,
        'data' => $product
    ]);
}
```

Gambar 05: Fungsi Show pada ProductsController

```
// ◆ POST /products
public function store(Request $request)
{
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);

    $product = Product::create($request->all());

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil ditambahkan',
        'data' => $product
    ]);
}
```

Gambar 06: Fungsi Store pada ProductsController

```
// ◆ PUT /products/{id}
public function update(Request $request, $id)
{
    $product = Product::find($id);
    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);
    $product->update($request->all());
    return response()->json([
        'status' => true,
        'message' => 'Product berhasil diupdate',
        'data' => $product
    ]);
}
```

Gambar 07: Fungsi Update pada ProductsController

```
// ◆ DELETE /products/{id}
public function destroy($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    $product->delete();

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil dihapus'
    ]);
}
```

Gambar 08: Fungsi Destroy pada ProductsController

E. API Documentation

1. Install Scramble dengan menggunakan composer
`composer require dedoc/scramble`
2. Masukkan ke vendor laravel project anda
`php artisan vendor:publish`
`--provider="Dedoc\Scramble\ScrambleServiceProvider"`
3. Jalankan server (artisan serve)
4. Akses API documentation pada browser ke <http://localhost:8000/docs/api>
5. Untuk menambahkan deksripsi setiap API Endpoints bisa menggunakan DocBlock diatas fungsi dalam controller
6. Menambahkan DocBlock (Title dan Description)

```

class ProductController extends Controller
{
  /**
   * Ambil semua produk.
   *
   * Endpoint ini mengembalikan seluruh data produk yang tersedia.
   */

  // ♦ GET /products
  public function index()
  {
    $products = Product::all();

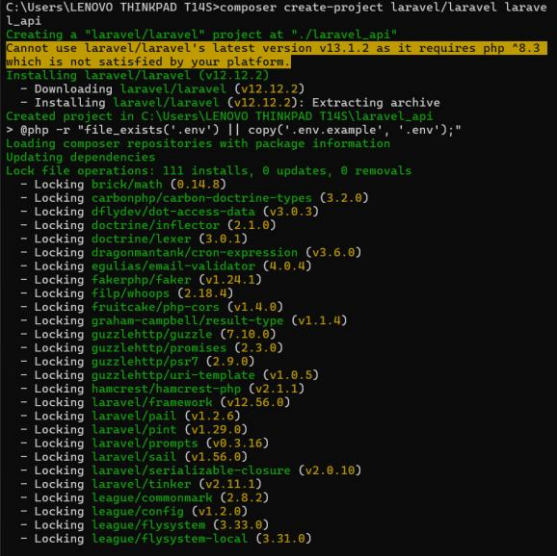
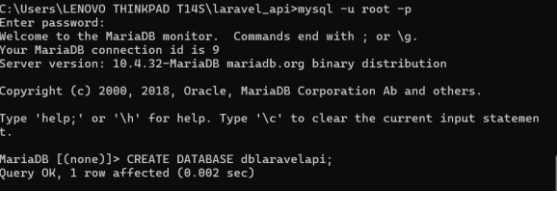
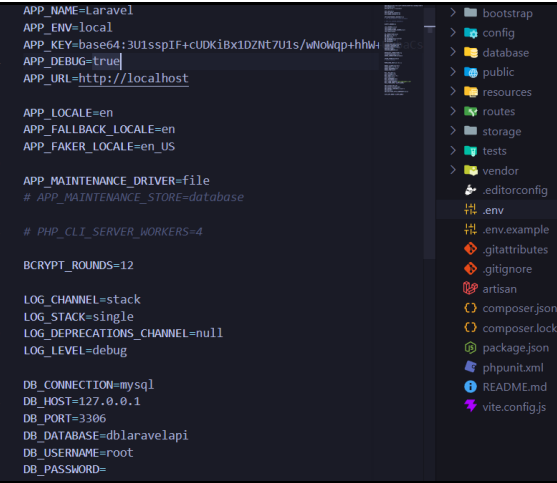
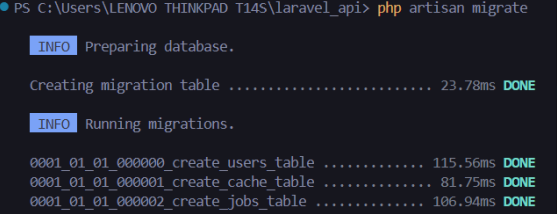
    return response()->json([
      'status' => true,
      'data' => $products
    ]);
  }
}

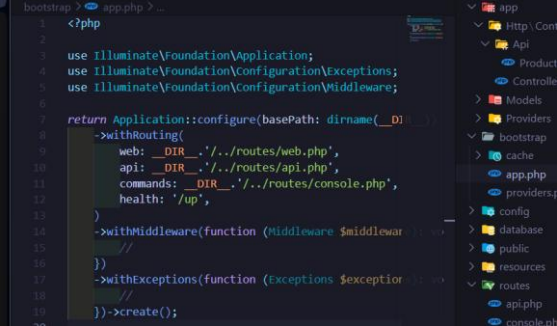
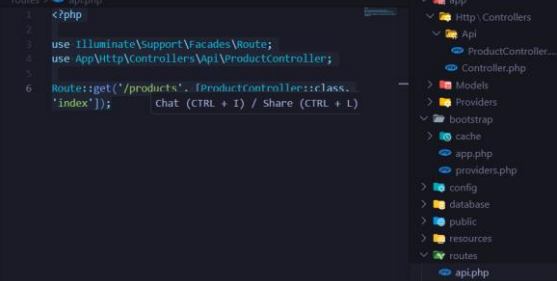
```

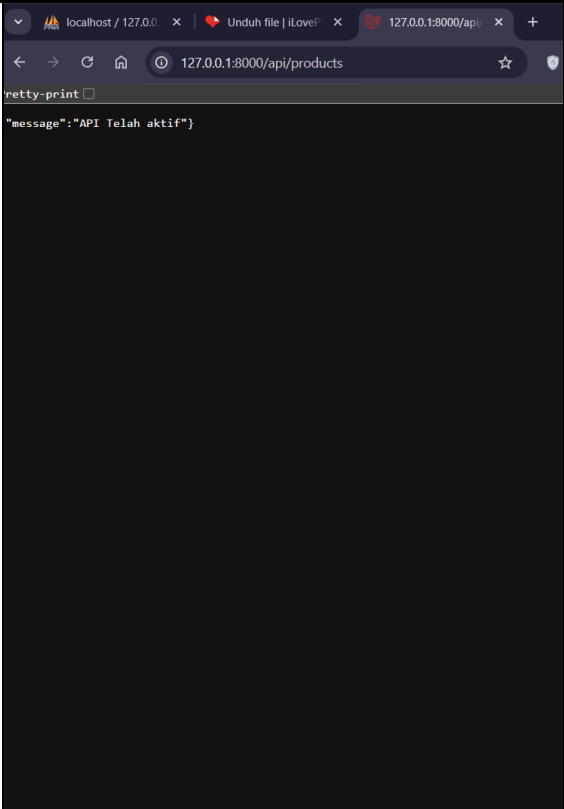
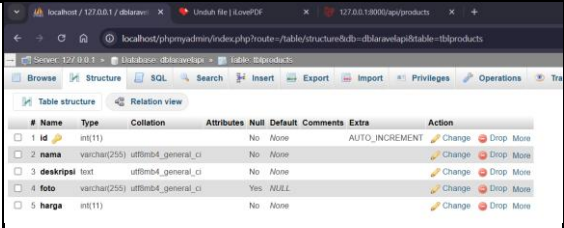
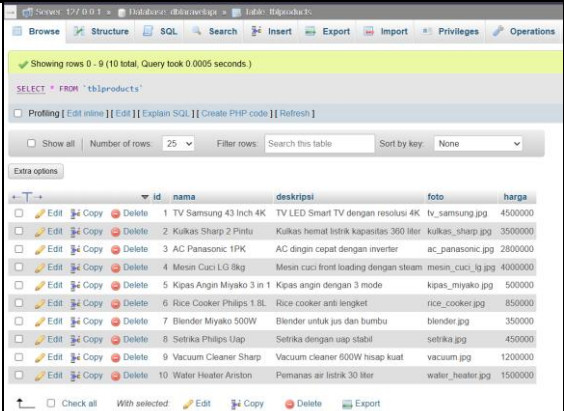
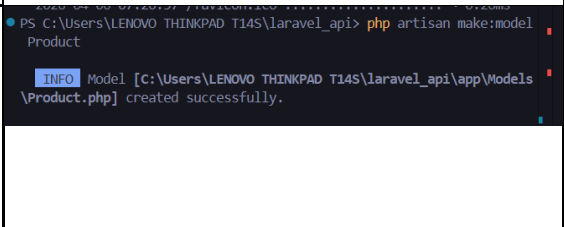
Gambar 08: DocBlock untuk title node index dan deksripsi

7. Refresh browser ke <http://localhost:8000/docs/api>
8. Tambahkan Title dan Deskripsi pada semua Endpoints

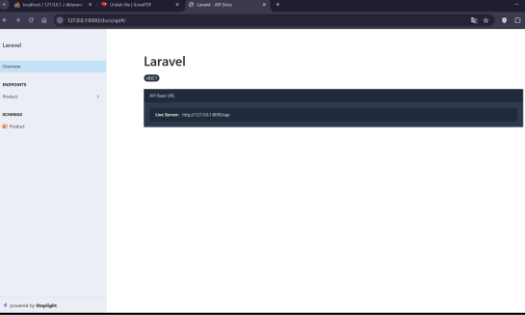
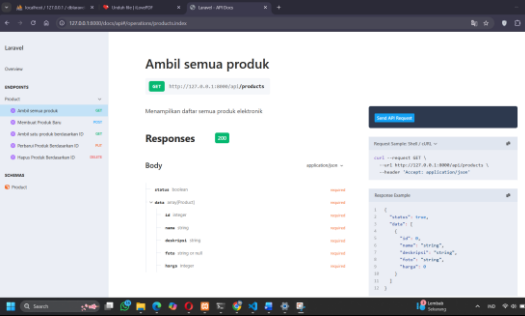
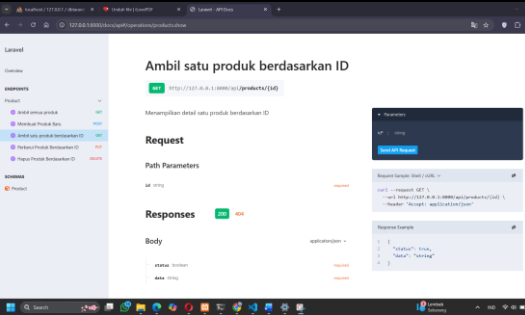
Hasil Pengerjaan

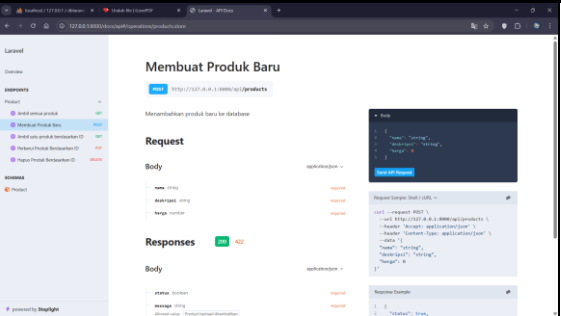
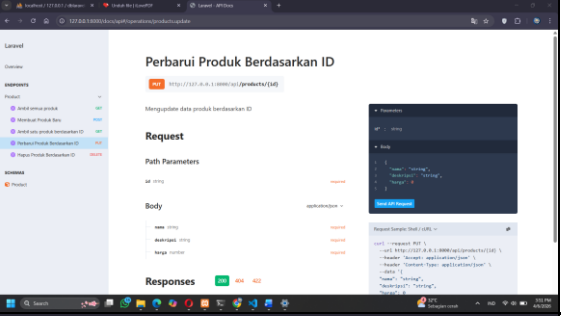
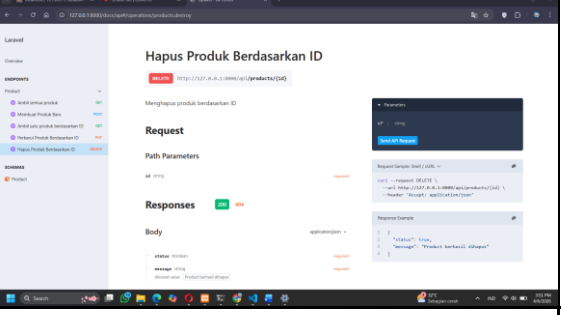
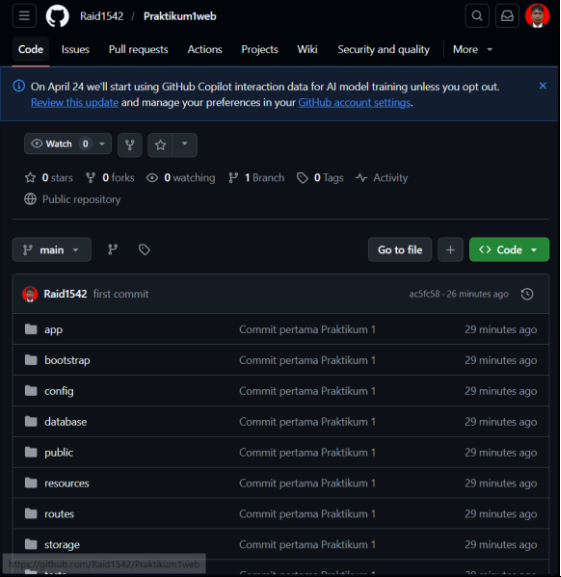
No.	Instruksi	Screenshot	Kendala/Saran
A.	Instalasi dan Konfigurasi		
1.	Buat Projek Baru		-
2.	Mempersiapkan Database		-
3.	Menghubungkan laravel dengan database yang dibuat		-
4.	Melakukan migrasi (php artisan migrate)		-

5.	Mengecek untuk memastikan laravel sudah terhubung dengan tabel yang dibuat	<pre> MariaDB [dblaravelapi]> show tables; +-----+ Tables_in_dblaravelapi +-----+ cache cache_locks failed_jobs job_batches jobs migrations password_reset_tokens sessions users +-----+ 9 rows in set (0.001 sec) </pre>	-
6.	Membuat controller Api/ProductController	<pre> PS C:\Users\LENOVO THINKPAD T14S\laravel_api> php artisan make:controller Api/ProductController [INFO] Controller [C:\Users\LENOVO THINKPAD T14S\laravel_api\app\Http\Controllers\Api\ProductController.php] created successfully. </pre>	-
7.	Menambah kode __DIR__ api: __DIR__.'../routes/api.php', pada bootstrap/app.php		-
8.	Membuat Routes baru di router/api.php		-

9.	Test API		-
10.	Membuat tabel tblproducts		-
11.	Membuat 10 data dengan nama product elektronik		-
12.	Membuat model bernama Product		-

13.	Menghubungkan model Product dengan tblproduct	<pre><?php namespace App\Models; use Illuminate\Database\Eloquent\Model; class Product extends Model { protected \$table = 'tblproducts'; protected \$primaryKey = 'id'; public \$timestamps = false; //Field yang boleh diisi protected \$fillable = ['nama', 'deskripsi', 'foto', 'harga',]; }</pre>	-
14.	Edit fungsi index pada Product Controller untuk dapatkan data dari model Product	<pre><?php namespace App\Http\Controllers\Api; use App\Http\Controllers\Controller; use Illuminate\Http\Request; use App\Models\Product; class ProductController extends Controller { public function index() { \$products = Product::all(); return response()->json(['data' => \$products]); } }</pre>	-
15.	Test API		-
16.	Ubah api.php dengan api Resource	<pre><?php use Illuminate\Support\Facades\Route; use App\Http\Controllers\Api\ProductController; // Route::get('/products', // [ProductController::class, 'index']); Route::apiResource('products', ProductController::class);</pre>	-
17.	Mengubah ProductController untuk fungsi-fungsi berikut index, show, store, update, dan destroy	<pre><?php namespace App\Http\Controllers\Api; use App\Http\Controllers\Controller; use Illuminate\Http\Request; use App\Models\Product; class ProductController extends Controller { // GET /products public function index() { \$products = Product::all(); return response()->json(['status' => true, 'data' => \$products]); } // GET /products/{id} public function show(\$id) { \$product = Product::find(\$id); if (!\$product) { return response()->json(['status' => false, 'message' => 'Product tidak ditemukan'], 404); } return response()->json(['status' => true, 'data' => \$product]); } }</pre>	-

18.	Install scramble dengan menggunakan composer	<pre> C:\Users\LENOVO THINKPAD T14S\laravel_api>composer require dedoc/scramble ./composer.json has been updated Running composer update dedoc/scramble Loading composer repositories with package information Updating dependencies Lock file operations: 3 installs, 0 updates, 0 removals - Locking dedoc/scramble (v0.13.17) - Locking phpstan/phpdoc-parser (2.3.2) - Locking spatie/laravel-package-tools (1.93.0) Writing lock file Installing dependencies from lock file (including require-dev) Package operations: 3 installs, 0 updates, 0 removals - Downloading spatie/laravel-package-tools (1.93.0) - Downloading phpstan/phpdoc-parser (2.3.2) - Downloading dedoc/scramble (v0.13.17) - Installing spatie/laravel-package-tools (1.93.0): Extracting archive - Installing phpstan/phpdoc-parser (2.3.2): Extracting archive - Installing dedoc/scramble (v0.13.17): Extracting archive Generating optimized autoload files > Illuminate\Foundation\ComposerScripts::postAutoloadDump > @php artisan package:discover --ansi INFO Discovering packages. dedoc/scramble DONE laravel/pail DONE laravel/sail DONE laravel/tinker DONE nesbot/carbon DONE nunomaduro/collision DONE nunomaduro/termwind DONE 83 packages you are using are looking for funding. Use the 'composer fund' command to find out more! > @php artisan vendor:publish --tag=laravel-assets --ansi --force </pre>	-
19	Masukan ke vendor laravel project	<pre> C:\Users\LENOVO THINKPAD T14S\laravel_api>php artisan vendor:publish --provi der="Dedoc\Scramble\ScrambleServiceProvider" INFO Publishing assets. Copying file [C:\Users\LENOVO THINKPAD T14S\laravel_api\vendor\dedoc\scram ble\config\scramble.php] to [C:\Users\LENOVO THINKPAD T14S\laravel_api\confi g\scramble.php] DONE Copying directory [C:\Users\LENOVO THINKPAD T14S\laravel_api\vendor\dedoc\ scramble\resources\views] to [C:\Users\LENOVO THINKPAD T14S\laravel_api\reso urces\views\vendor\scramble] DONE </pre>	-
20.	Akses API documentation http://localhost:8000/docs/api		-
21.	API documentation index		-
22.	API documentation show		-

23.	API documentation store		-
24.	API documentation update		-
25.	API documentation destroy		-
B.	Github dan Viscode		
1.	Commit tugas praktikum ke github		-